

Kadane Algorithm Complete DSA Notes - Leetcode 53 Maximum Subarray

1. What is Kadane Algorithm?

Kadane Algorithm finds the maximum sum contiguous subarray.

It is a Dynamic Programming optimization.

Example:

`[-2,1,-3,4,-1,2,1,-5,4]`

Best Subarray:

`[4,-1,2,1]`

Answer = 6

Used in:

- Arrays
- DP
- Optimization problems

2. Core Intuition

At every element:

Question:

Should I extend my previous subarray?

OR

Start a new subarray?

If previous sum becomes negative,
remove it because it decreases future answers.

3. DP Formula

State:

currentSum =
maximum subarray ending at index i

Transition:

currentSum =
max(nums[i],
currentSum + nums[i])

Answer:

maximum currentSum seen.

4. Brute Force

Generate every subarray.

Two loops.

Complexity:

Time:
 $O(n^2)$

Space:
 $O(1)$

5. Kadane Optimal Approach

Steps:

1. Maintain running sum.
2. Add current element.
3. Update maximum.
4. If sum < 0:
discard it.

Complexity:

Time:
 $O(n)$

Space:
O(1)

Python Implementation

```
def maxSubArray(nums):  
    maximum = nums[0]  
    current = 0  
  
    for x in nums:  
        current += x  
  
        maximum=max(  
            maximum,  
            current  
        )  
  
        if current < 0:  
            current = 0  
  
    return maximum
```

C++ Implementation

```
int maxSubArray(vector<int>& nums){  
    int sum=0;  
    int ans=nums[0];  
  
    for(int x:nums){  
        sum+=x;  
  
        ans=max(ans, sum);  
  
        if(sum<0)  
            sum=0;  
    }  
  
    return ans;  
}
```

Java Implementation

```
class Solution{  
    public int maxSubArray(int[] nums){  
        int sum=0;  
        int ans=nums[0];  
        for(int x:nums){
```

```

        sum+=x;

        ans=Math.max(ans, sum);

        if(sum<0)
            sum=0;
    }

    return ans;
}
}

```

6. Printing Maximum Subarray

Modification:

Track:

start index

ending index

temporary start

Whenever current sum becomes negative:
move starting position.

Python Print Subarray

```

def printSubarray(nums):
    start=0
    ansStart=0
    ansEnd=0

    total=0
    best=nums[0]

    for i in range(len(nums)):
        total+=nums[i]

        if total>best:
            best=total
            ansStart=start
            ansEnd=i

        if total<0:
            total=0
            start=i+1
    return nums[ansStart:ansEnd+1]

```

7. All Negative Case

Important interview edge case:

[-8,-3,-5]

Answer:

-3

Initialize answer with nums[0],
not zero.

8. Kadane Variations

Important Leetcode Patterns:

LC 53:
Maximum Subarray

LC 918:
Maximum Circular Subarray

LC 152:
Maximum Product Subarray

LC 121:
Best Time Buy Sell Stock

Advanced:

Maximum Sum Rectangle in Matrix

9. Circular Kadane

Circular Array:

Two possibilities:

1. Normal maximum subarray
2. Wrapping subarray

Formula:

```
max(  
normalKadane,  
totalSum - minimumSubarray  
)
```

10. Maximum Product Subarray

Maintain:

maximum product

minimum product

Reason:

Negative * Negative can become maximum.

Pattern Recognition

Use Kadane when problem says:

- maximum continuous
- minimum continuous
- contiguous subarray
- best segment

Think:

Can previous answer help current index?